

Internal Services with SSL

A common use case you might have is to host services and applications which are only reachable within your clan.

This guide explains how to set up such secure, clan-internal web services using a custom top-level domain (TLD) with SSL certificates.

Your services will be accessible only within your clan network and secured with proper SSL certificates that all clan machines trust.

Overview

By combining the `coredns` and `certificates` clan services, you can:

- Create a custom TLD for your clan (e.g. `.c`)
- Host internal web services accessible via HTTPS (e.g. `https://api.c`, `https://dashboard.c`)
- Automatically provision and trust SSL certificates across all clan machines
- Keep internal services secure and isolated from the public internet

The setup uses two clan services working together:

- **coredns service:** Provides DNS resolution for your custom TLD within the clan
- **certificates service:** Creates a certificate authority (CA) and issues SSL certificates for your TLD

DNS Resolution Flow

1. A clan machine tries to access `https://service.c`
2. The machine queries its local DNS resolver (unbound)

3. For `.c` domains, the query is forwarded to your clan's CoreDNS server. All other domains will be resolved as usual.
4. CoreDNS returns the IP address of the machine hosting the service
5. The machine connects directly to the service over HTTPS
6. The SSL certificate is trusted because all machines trust your clan's CA

Step-by-Step Setup

The following setup assumes you have a VPN (e.g. ZeroTier) already running. The IPs configured in the options below will probably be the ZeroTier IPs of the respective machines.

Configure the CoreDNS Service

The CoreDNS service has two roles:

- `server`: Runs the DNS server for your custom TLD
- `default`: Makes machines use the DNS server for TLD resolution and allows exposing services

Add this to your inventory:

```
1 inventory = {
2   machines = {
3     dns-server = { }; # Machine that will run the DNS server
4     web-server = { }; # Machine that will host web services
5     client = { };     # Any other machines in your clan
6   };
7
8   instances = {
9     coredns = {
10
11       # Add the default role to all machines
12       roles.default.tags = [ "all" ];
13
14       # DNS server for the .c TLD
15       roles.server.machines.dns-server.settings = {
16         ip = "192.168.1.10"; # IP of your DNS server machine
17         tld = "c";
18       };
19
20       # Machine hosting services (example: ca.c and admin.c)
21       roles.default.machines.web-server.settings = {
```

```

22     ip = "192.168.1.20"; # IP of your web server
23     services = [ "ca" "admin" ];
24 };
25 };
26 };
27 };

```

Configure the Certificates Service

The certificates service also has two roles:

- `ca`: Sets up the certificate authority on a server
- `default`: Makes machines trust the CA and allows them to request certificates

Add this to your inventory:

```

1 inventory = {
2   instances = {
3     # ... coredns configuration from above ...
4
5     certificates = {
6
7       # Set up CA for .c domain
8       roles.ca.machines.dns-server.settings = {
9         tlds = [ "c" ];
10        acmeEmail = "admin@example.com"; # Optional: your email
11      };
12
13      # Add default role to all machines to trust the CA
14      roles.default.tags = [ "all" ];
15    };
16  };
17 };

```

Complete Example Configuration

Here's a complete working example:

```

1 inventory = {
2   machines = {

```

```
3  caserver = { }; # DNS server + CA + web services
4  webserver = { }; # Additional web services
5  client = { }; # Client machine
6  };
7
8  instances = {
9    coredns = {
10
11      # Add the default role to all machines
12      roles.default.tags = [ "all" ];
13
14      # DNS server for the .c TLD
15      roles.server.machines.caserver.settings = {
16        ip = "192.168.8.5";
17        tld = "c";
18      };
19
20      # machine hosting https://ca.c (our CA for SSL)
21      roles.default.machines.caserver.settings = {
22        ip = "192.168.8.5";
23        services = [ "ca" ];
24      };
25
26      # machine hosting https://blub.c (some internal web-service)
27      roles.default.machines.webserver.settings = {
28        ip = "192.168.8.6";
29        services = [ "blub" ];
30      };
31    };
32
33    # Provide https for the .c top-level domain
34    certificates = {
35
36      roles.ca.machines.caserver.settings = {
37        tlds = [ "c" ];
38        acmeEmail = "admin@example.com";
39      };
40
41      roles.default.tags = [ "all" ];
42    };
43  };
44  };
```

Testing Your Configuration

DNS resolution can be tested with:

```
# On any clan machine, test DNS resolution
nslookup ca.c
nslookup blub.c
```

You should also now be able to visit `https://ca.c` to access the certificate authority or visit `https://blub.c` to access your web service.

Troubleshooting

DNS Resolution Issues

1. Check if DNS server is running:

```
# On the DNS server machine
systemctl status coredns
```

2. Verify DNS configuration:

```
# Check if the right nameservers are configured
cat /etc/resolv.conf
systemctl status systemd-resolved
```

3. Test DNS directly:

```
# Query the DNS server directly
dig @192.168.8.5 ca.c
```

Certificate Issues

1. Check CA status:

```
# On the CA machine
```

```
systemctl status step-ca  
systemctl status nginx
```

2. Verify certificate trust:

```
# Test certificate trust  
curl -v https://ca.c  
openssl s_client -connect ca.c:443 -verify_return_error
```

3. Check ACME configuration:

```
# View ACME certificates  
ls /var/lib/acme/  
journalctl -u acme-ca.c.service
```

← Previous

Exports

Next →

Introduction to Vars

Need help? Reach out to [our community](#).